The Journal of Open Source Software

AutoEncoderToolkit.jl: A Julia package for training (Variational) Autoencoders

Manuel Razo-Mejia [1,2]

1 Department of Biology, Stanford University, CA, United States of America **2** For correspondence, contact mrazo@stanford.edu

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Fabian Scheipl

Reviewers:

- @sandeshkatakam
- @avik-pal

Submitted: 10 May 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

With the advent of generative models, the field of unsupervised learning has exploded in the last decade. One of the most popular generative models is the variational autoencoder (VAE) (Kingma & Welling, 2014). VAEs assume the existence of a joint probability distribution between a high-dimensional data space and a lower-dimensional latent space. The VAE parametrizes this joint distribution with two neural networks–an encoder and a decoder–using a variational inference approach. This approach allows for the model to approximate the underlying low-dimensional structure that generated the observed data and generate new data samples by sampling from the learned latent space. Several variations of the original VAE have been proposed to extend its capabilities and tackle different problems. Here, we present `AutoEncoderToolkit.jl`, a `Julia` package for training VAEs and its extensions. The package is built on top of the `Flux.jl` deep learning library (Innes, 2018) and provides a simple and flexible interface for training different flavors of VAEs. Furthermore, the package provides a set of utilities for the geometric analysis of the learned latent space.

Statement of need

The collection and analysis of large high-dimensional datasets have become routine in several scientific fields. Therefore, the need to understand and uncover the underlying low-dimensional structure of these datasets is more pressing than ever. VAEs, have shown great promise in this regard, with applications ranging from single-cell transcriptomics (Lopez, 2018), to protein design (Lian et al., 2022), to the discovery of the governing equations of dynamical systems (Champion et al., 2019). However, most of these tools have been exclusively developed for the `Python` ecosystem. `Julia` is a promising high-performance language for scientific computing, with a growing ecosystem for deep learning, state-of-the-art automatic differentiation, and a strong GPU-accelerated computing backend. Furthermore, the programming paradigm of `Julia`, based on multiple dispatch, allows for a more flexible, modular, and composable codebase. `AutoEncoderToolkit.jl` aims to provide a simple and flexible interface for training VAEs and its extensions in `Julia` taking full advantage of the programming paradigm of the language.

Software Description

Encoder & Decoder definition

`AutoEncoderToolkit.jl` takes a probability-theory-based approach to the design of the package. Taking advantage of the multiple dispatch paradigm of `Julia`, the set of encoders and decoders provided by the package can be easily combined and extended to quickly prototype different

VAE architectures. In other words, independent of the design of the multi-layer perceptron used for either the encoder or the decoder, what defines their behavior is their associated probability distribution. For example, let us consider the loss function for the standard VAE model, given by the evidence lower bound (ELBO):

$$\text{ELBO} = \langle \log p_\theta(x|z) \rangle_{q_\phi(z|x)} - D_{KL}(q_\phi(z|x)||p(z)), \tag{1}$$

where $p_\theta(x|z)$ is the likelihood of the data given the latent variable defined by the decoder with parameters θ, $q_\phi(z|x)$ is the posterior distribution of the latent variable given the data defined by the encoder with parameters ϕ, and $D_{KL}(q_\phi(z|x)||p(z))$ is the Kullback-Leibler divergence between the posterior and the prior distribution of the latent space. By defining the decoder type as either `BernoulliDecoder` or `SimpleGaussianDecoder`, the user can decide whether decoder outputs, i.e., the parameters of the likelihood distribution $p_\theta(x|z)$, are the parameters of a Bernoulli distribution or a Gaussian distribution with constant diagonal covariance, respectively. This design choice allows for the quick prototyping of different architectures without the overhead of defining new specific losses for each type of decoder.

Furthermore, the design allows for the easy extension of the list of available encoders and decoders that can directly integrate to any of the existing list of VAE models. For example, let us assume that for a particular problem, the user wants to define a decoder whose output are the parameters for independent Poisson distributions, each with a different parameter λ_i. In other words, on the decoder side, the decoder returns a vector of parameters λ for each of the dimensions of the data. The user can define a new decoder type

```julia
struct PoissonDecoder <: AbstractVariationalDecoder
    decoder::Flux.Chain
end # struct
```

With this `struct` defined, the user only needs to define two methods: one for for the forward pass of the decoder

```julia
function (decoder::PoissonDecoder)(z::AbstractArray)
    # Run input to decoder network
    return (λ=decoder.decoder(z),)
end # function
```

and another for the likelihood of the data given the latent variable

```julia
function decoder_loglikelihood(
        x::AbstractArray,
        z::AbstractVector,
        decoder::PoissonDecoder,
        decoder_output::NamedTuple;
)
        # Extract the lambda parameter of the Poisson distribution
        λ = decoder_output.λ

        # Compute log-likelihood
        loglikelihood = sum(x .* log.(λ) - λ - loggamma.(x .+ 1))

        return loglikelihood
end # function
```

where we use the log-likelihood function of multiple independent Poisson distributions, given by

$$\ln p(x|\lambda) = \sum_i x_i \log(\lambda_i) - \lambda_i - \log(\Gamma(x_i + 1)). \tag{2}$$

Wit these methods defined, the `PoissonDecoder` can be directly integrated into any of the different VAE models provided by the package.

Implemented models

At the time of this writing, the package has implemented the following models:

Table 1: List of implemented (variational) autoencoder models

Name	Reference
Deterministic Autoencoder	
Vanilla Variational Autoencoder (VAE)	(Kingma & Welling, 2014)
β-Variational Autoencoder (β-VAE)	(Higgins et al., 2017)
Maximum-Mean Discrepancy Variational Autoencoder (InfoVAE)	(Zhao et al., 2018)
InfoMax Variational Autoencoder (InfoMaxVAE)	(Rezaabad & Vishwanath, 2020)
Hamiltonian Variational Autoencoder (HVAE)	(Caterini et al., 2018)
Riemannian Hamiltonian Variational Autoencoder (RHVAE)	(Chadebec et al., 2020)

Other than the deterministic autoencoder, all the models listed above use the same underlying `VAE` struct as part of their definition. Some of them, like the `InfoMaxVAE` and the `RHVAE`, require additional neural networks used during training or inference. However, other than those additional elements, the training routines for all models is virtually the same. This design choice allows the user to quickly explore different VAE models for their specific applications without the need to write new training routines for each model.

Moreover, extending the list of VAE models is also straightforward as contributions to the package only need to focus on a general definition of the loss function associated with the new VAE model, without the need to define specific terms for each type of encoder or decoder.

Differential geometry utilities

In recent years, there has been a growing interest in understanding the geometric properties of the latent space learned by VAEs Arvanitidis et al. (2021). This is because the non-linearities of the encoder and decoder networks can induce complex geometries in the latent space, where the Euclidean distance between points in latent space does not necessarily reflect the true distance between the corresponding data points. Thus, tools from differential geometry such as geodesic distances, parallel transport, and curvature can provide deeper insights into the structure of the learned latent space. `AutoEncoderToolkit.jl` provides a set of utilities for the geometric analysis of the latent space. For example, at the time of this writing, the `NeuralGeodedics` module provides the tools to approximate geodesic curves between points in latent space for the Riemannian Hamiltonian VAE (RHVAE) model. This is achieved by utilizing a neural network to approximate the geodesic equation (Chen et al., 2018) in the latent space using the explicit representation of the Riemannian metric learned by the RHVAE model (Chadebec et al., 2020).

GPU support

`AutoEncoderToolkit.jl` offers GPU support for `CUDA.jl`-compatible GPUs out of the box.

Documentation

Documentation is available at (https://mrazomej.github.io/AutoEncoderToolkit.jl), where there are worked out examples and tutorials on how to use the package.

DRAFT

Acknowledgements

I would like to thank Griffin Chure, Madhav Mani, and Dmitri Petrov for their helpful advice and discussions during the development of this package. I would also like to thank the Schmidt Science Fellows program for funding part of this work via a postdoctoral fellowship.

References

Arvanitidis, G., Hansen, L. K., & Hauberg, S. (2021, December 13). *Latent Space Oddity: On the Curvature of Deep Generative Models*. http://arxiv.org/abs/1710.11379

Caterini, A. L., Doucet, A., & Sejdinovic, D. (2018). *Hamiltonian Variational Auto-Encoder*. 11.

Chadebec, C., & Allassonnière, S. (2022, November 3). *A Geometric Perspective on Variational Autoencoders*. http://arxiv.org/abs/2209.07370

Chadebec, C., Mantoux, C., & Allassonnière, S. (2020, October 22). *Geometry-Aware Hamiltonian Variational Auto-Encoder*. http://arxiv.org/abs/2010.11518

Champion, K., Lusch, B., Kutz, J. N., & Brunton, S. L. (2019). Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, *116*(45), 22445–22451. https://doi.org/10.1073/pnas.1906995116

Chen, N., Klushyn, A., Kurle, R., Jiang, X., Bayer, J., & Smagt, P. (2018). Metrics for Deep Generative Models. *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, 1540–1550. https://proceedings.mlr.press/v84/chen18e.html

Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., & Lerchner, A. (2017). *B-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*.

Innes, M. (2018). Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, *3*(25), 602. https://doi.org/10.21105/joss.00602

Kingma, D. P., & Welling, M. (2014, May 1). *Auto-Encoding Variational Bayes*. http://arxiv.org/abs/1312.6114

Lian, X., Praljak, N., Subramanian, S. K., Wasinger, S., Ranganathan, R., & Ferguson, A. L. (2022). *Deep learning-enabled design of synthetic orthologs of a signaling protein* [Preprint]. Molecular Biology. https://doi.org/10.1101/2022.12.21.521443

Lopez, R. (2018). Deep generative modeling for single-cell transcriptomics. *Nature Methods*, *15*.

Rezaabad, A. L., & Vishwanath, S. (2020, January 7). *Learning Representations by Maximizing Mutual Information in Variational Autoencoders*. http://arxiv.org/abs/1912.13361

Zhao, S., Song, J., & Ermon, S. (2018, May 30). *InfoVAE: Information Maximizing Variational Autoencoders*. http://arxiv.org/abs/1706.02262

DRAFT